



Mini-Curso de Python

Do Básico ao Avançado

Agenda do Mini-Curso

Dia 1: Nível Básico

- ✓ Introdução ao Python
- ✓ Lógica de Programação
- ✓ Tipos de Dados
- ✓ Variáveis
- ✓ Operadores
- ✓ Entrada e Saída de Dados

Dia 2: Nível Intermediário

- ✓ Estruturas de Dados
- ✓ Listas, Tuplas, Conjuntos
- ✓ Dicionários
- ✓ Controle de Fluxo (if, elif, else)
- ✓ Estruturas de Repetição (for, while)
- ✓ Tratamento de Erros (try, except)

Dia 3: Nível Avançado

- ✓ Funções
- ✓ Programação Orientada a Objetos
- ✓ Classes e Objetos
- ✓ Encapsulamento
- ✓ Herança
- ✓ Polimorfismo e Abstração

Dia 1: Introdução ao Python

O que é Python?

Linguagem de programação de alto nível, criada em 1991 por Guido van Rossum, muito utilizada em diversas áreas como ciência de dados, inteligência artificial, engenharia, desenvolvimento web e automação.

Por que usar Python?

- ★ **Sintaxe simples e legível**- código limpo e fácil de entender
- ★ **Multiplataforma**- funciona em Windows, Linux, Mac
- ★ **Linguagem interpretada**- não precisa compilar o código
- ★ **Grande comunidade**- fácil encontrar ajuda e recursos
- ★ **Muitas bibliotecas**- para praticamente qualquer tarefa



Empresas que usam Python:

Google

Netflix

NASA

Spotify

Dia 1: Fundamentos de Programação

Lógica de Programação

A lógica de programação é a organização coerente de instruções para resolver problemas. Em Python, escrevemos instruções de forma clara e legível.

Variáveis

Variáveis são espaços na memória que armazenam dados. Em Python, não precisamos declarar o tipo.

```
nome = 'Maria'
idade = 25
altura = 1.65
estudante = True
```

Tipos de Dados

 str	'Python', "Olá, mundo!"
 int	42, -10, 0
 float	3.14, -0.5, 2.0
 bool	True, False

Operadores

```
# Aritméticos
soma = a + b
subtracao = a - b
multiplicacao = a * b
divisao = a / b

# Comparação
igual = a == b
maior = a > b
menor_igual = a <= b
```

Dia 1: Entrada e Saída de Dados

Função print()

A função `print()` exibe informações na tela. É uma das funções mais utilizadas em Python.

```
print('Olá, mundo!')
print('Nome:', 'Maria')
idade = 25
print(f'Idade: {idade} anos')
```

```
Olá, mundo!
Nome: Maria
Idade: 25 anos
```

Dica:

Use f-strings (`f'texto {variavel}'`) para formatar texto com variáveis de forma simples e legível.

Função input()

A função `input()` captura dados digitados pelo usuário. Sempre retorna uma string.

```
nome = input('Digite seu nome: ')
print(f'Olá, {nome}!')
```



```
# Convertendo para número
idade = int(input('Digite sua idade: '))
print(f'Daqui a 5 anos você terá {idade + 5} anos')
```

```
Digite seu nome: Carlos
Olá, Carlos!
Digite sua idade: 30
Daqui a 5 anos você terá 35 anos
```

Importante:

Use `int()`, `float()` ou `bool()` para converter a entrada para o tipo desejado.

Dia 2: Estruturas de Dados

Python possui diversas estruturas de dados nativas que facilitam o armazenamento e manipulação de informações:

Listas (Lists)

Coleção ordenada e mutável de elementos

```
lista = [1, 2, 3, 'Python']
```

Tuplas (Tuples)

Coleção ordenada e imutável de elementos

```
tupla = (1, 2, 3, 'Python')
```

Conjuntos (Sets)

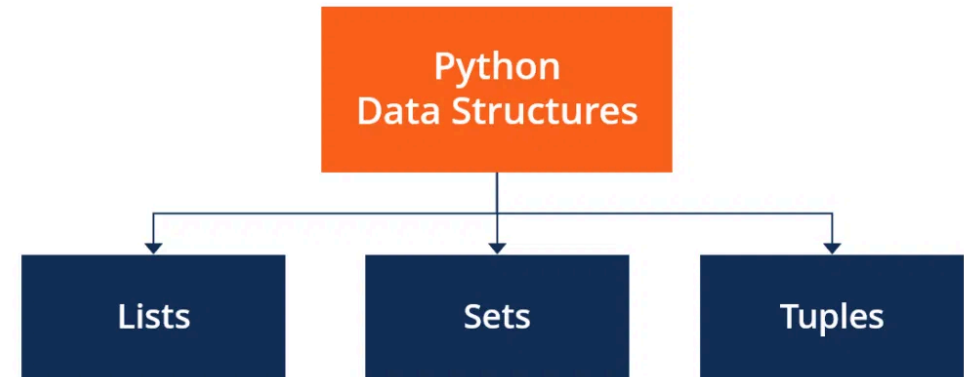
Coleção não ordenada de elementos únicos

```
conjunto = {1, 2, 3, 'Python'}
```

Dicionários (Dictionaries)

Coleção de pares chave-valor

```
dicionario = {'nome': 'Python', 'ano': 1991}
```



Dia 2: Controle de Fluxo

Estruturas Condicionais

Permitem que o programa tome decisões e execute diferentes blocos de código com base em condições.

```
# Estrutura if-elif-else
nota = 7.5

if nota >= 7:
    print("Aprovado")
elif nota >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

Dica

Em Python, a indentação é obrigatória e define os blocos de código. Use 4 espaços para cada nível de indentação.

Operadores de Comparação

== - Igual a
!= - Diferente de
> - Maior que
< - Menor que
>= - Maior ou igual a
<= - Menor ou igual a

Operadores Lógicos

and - E lógico (ambas condições verdadeiras)
or - OU lógico (pelo menos uma condição verdadeira)
not - NÃO lógico (inverte o valor)

```
# Combinando operadores
idade = 25
tem_carteira = True

if idade >= 18 and tem_carteira:
    print("Pode dirigir")
else:
    print("Não pode dirigir")
```

Dia 2: Estruturas de Repetição

Loop For

Utilizado para iterar sobre uma sequência (lista, tupla, string) ou outros objetos iteráveis.

```
# Iterando sobre uma lista
frutas = ['maçã', 'banana', 'laranja']
for fruta in frutas:
    print(f'Eu gosto de {fruta}')
```



```
# Usando range()
for i in range(1, 6):
    print(f'Contagem: {i}')
```

- ✓ Útil quando você sabe quantas vezes deseja repetir o código
- ✓ Pode usar `range()` para gerar sequências numéricas
- ✓ Pode usar `enumerate()` para obter índice e valor

Loop While

Executa um bloco de código enquanto uma condição for verdadeira.

```
# Contagem regressiva
contador = 5
while contador > 0:
    print(f'Contagem: {contador}')
    contador -= 1
```



```
# Loop com condição de saída
senha = ''
while senha != '123':
    senha = input('Digite a senha: ')
```

- ✓ Útil quando não se sabe quantas repetições serão necessárias
- ✓ Cuidado com loops infinitos (condição sempre verdadeira)
- ✓ Use `break` para sair do loop e `continue` para pular iterações

Dia 2: Tratamento de Erros

Try e Except

O tratamento de exceções permite que seu programa continue executando mesmo quando ocorrem erros.

```
try:
    # Código que pode gerar erro
    numero = int(input('Digite um número: '))
    resultado = 10 / numero
    print(f'10 dividido por {numero} é
{resultado}')
except ValueError:
    # Executa se ocorrer erro de valor
    print('Entrada inválida. Digite apenas
números.')
except ZeroDivisionError:
    # Executa se ocorrer divisão por zero
    print('Não é possível dividir por zero.')
except:
    # Captura qualquer outro erro
    print('Ocorreu um erro inesperado.')
```

Por que usar tratamento de erros?

- Evita que o programa pare de executar
- Melhora a experiência do usuário
- Facilita a depuração de problemas

Estrutura Completa

```
try:
    # Código que pode gerar erro
    arquivo = open('dados.txt', 'r')
    conteudo = arquivo.read()
except FileNotFoundError:
    # Executa se o arquivo não for encontrado
    print('Arquivo não encontrado.')
else:
    # Executa se não ocorrer nenhum erro
    print(f'Arquivo lido com sucesso!')
    arquivo.close()
finally:
    # Sempre executa, independente de erro
    print('Operação finalizada.')
```

Erros Comuns em Python

-  **SyntaxError:** Erro na sintaxe do código
-  **NameError:** Variável ou função não definida
-  **TypeError:** Operação com tipos incompatíveis
-  **IndexError:** Índice fora do intervalo da lista
-  **KeyError:** Chave não encontrada no dicionário

Dia 3: Funções em Python

O que são Funções?

Funções são blocos de código reutilizáveis que realizam uma tarefa específica. Elas ajudam a organizar o código, evitar repetição e tornar o programa mais modular.

Definindo Funções

```
# Função simples
def saudacao():
    print("Olá, mundo!")

# Função com parâmetros
def saudar_pessoa(nome):
    print(f"Olá, {nome}!")

# Função com retorno
def soma(a, b):
    return a + b

# Chamando as funções
saudacao()
saudar_pessoa("Maria")
resultado = soma(5, 3)
print(f"Soma: {resultado}")
```

Tipos de Funções

⚙️ Funções com Parâmetros Padrão

```
def potencia(base, expoente=2):
    return base ** expoente

print(potencia(5))      # 25 (52)
print(potencia(2, 3))   # 8 (23)
```

⚙️ Funções Lambda (anônimas)

```
# Função lambda para calcular o dobro
dobro = lambda x: x * 2

# Função lambda com múltiplos parâmetros
soma = lambda a, b: a + b

print(dobro(5))      # 10
print(soma(3, 7))    # 10
```

- 💡 As funções ajudam a dividir problemas complexos em partes menores e mais gerenciáveis.
- 💡 Funções bem nomeadas tornam o código mais legível e auto-documentado.

Dia 3: Programação Orientada a Objetos - Conceitos

A Programação Orientada a Objetos (POO) é um paradigma baseado no conceito de "objetos", que podem conter dados e código: dados na forma de campos (atributos) e código na forma de procedimentos (métodos).

Classe

Molde ou estrutura que define como os objetos serão criados. É o projeto que define atributos e métodos.

Objeto

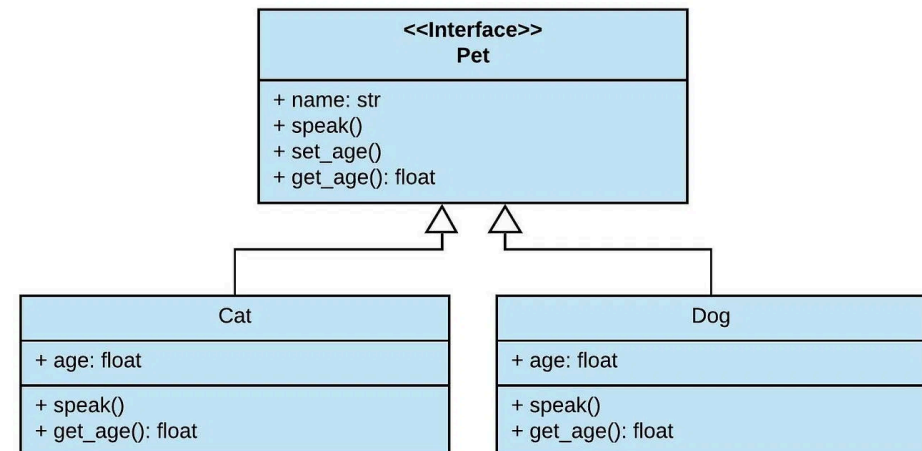
Instância concreta da classe, algo que existe na memória. É uma representação de uma entidade do mundo real.

Atributos

Características ou propriedades do objeto. São os dados armazenados dentro do objeto.

Métodos

Comportamentos ou ações que o objeto pode executar. São funções definidas dentro da classe.



Dia 3: POO Avançado - Herança e Polimorfismo

Herança

Permite que uma classe herde atributos e métodos de outra classe, promovendo o reuso de código.

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def fazer_som(self):
        print("Som genérico de animal")

class Cachorro(Animal):
    def fazer_som(self):
        print("Au au!")

rex = Cachorro("Rex")
rex.fazer_som()  # Saída: Au au!
```

Encapsulamento

Protege os dados de acessos indevidos e controla como atributos e métodos são acessados.

```
class Conta:
    def __init__(self, saldo):
        self.__saldo = saldo  # Atributo privado

    def depositar(self, valor):
        self.__saldo += valor

    def get_saldo(self):
        return self.__saldo
```



Polimorfismo

O mesmo método pode ter diferentes comportamentos em classes distintas. Permite tratar objetos de classes diferentes de maneira uniforme.

Abstração

Simplifica sistemas complexos, mostrando apenas o essencial. Em Python, pode ser implementada com classes abstratas usando o módulo ABC (Abstract Base Classes).

Vantagens da POO

- Organização do código em estrutura clara e modular
- Reutilização através de herança e composição
- Facilidade de manutenção com alterações localizadas
- Modelagem próxima ao mundo real